

End-to-End Machine Learning Project

Muhammad Umar

Working with Real Data?

Popular open data repositories:

- UC Irvine Machine Learning Repository
- Kaggle datasets
- Amazon AWS datasets
- Wikipedia List of Machine Learning Datasets
- Quora
- Subreddit datasets

Look at the big picture

The first task to perform is to build a model of housing price in California.

The model should learn from this data and be able to predict the median housing price in any district, given all the other metrics.

```
In [5]: housing = load_housing_data()  
housing.head()
```

Out[5]:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population
0	-122.23	37.88	41.0	880.0	129.0	322.0
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0
2	-122.24	37.85	52.0	1467.0	190.0	496.0
3	-122.25	37.85	52.0	1274.0	235.0	558.0
4	-122.25	37.85	52.0	1627.0	280.0	565.0

Frame the Problem

What the current solution looks like (if any). It will give us a reference performance, as well as insights on how to solve the problem.

We need to frame the problem

- (1) Is it supervised, unsupervised or Reinforcement Learning?
- (2) Is it a classification task, a regression task, or something else?
- (3) Should we use batch learning or online learning techniques?

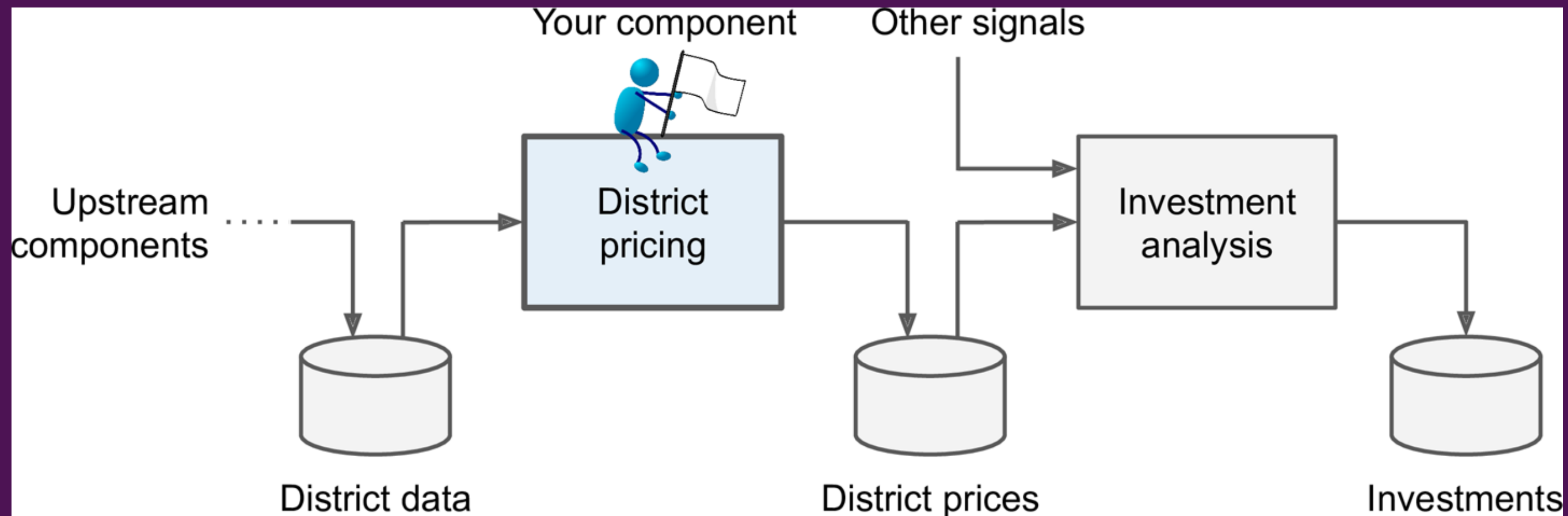


Fig: Machine Learning pipeline for real estate investments

Select a Performance Measure

A typical performance measure for regression problems is the Root Mean Square Error (RMSE).

It measures the standard deviation of the errors the system makes in the prediction.

In some contexts, we may prefer to use another function called Mean Absolute Error. (MAE)

Take a Quick look

info() method to get a quick description of data.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   longitude              20640 non-null  float64
1   latitude              20640 non-null  float64
2   housing_median_age     20640 non-null  float64
3   total_rooms            20640 non-null  float64
4   total_bedrooms         20433 non-null  float64
5   population             20640 non-null  float64
6   households              20640 non-null  float64
7   median_income          20640 non-null  float64
8   median_house_value     20640 non-null  float64
9   ocean_proximity        20640 non-null  object  
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

Fig: Housing dataset info

hist() method will plot histogram for each numerical attribute.

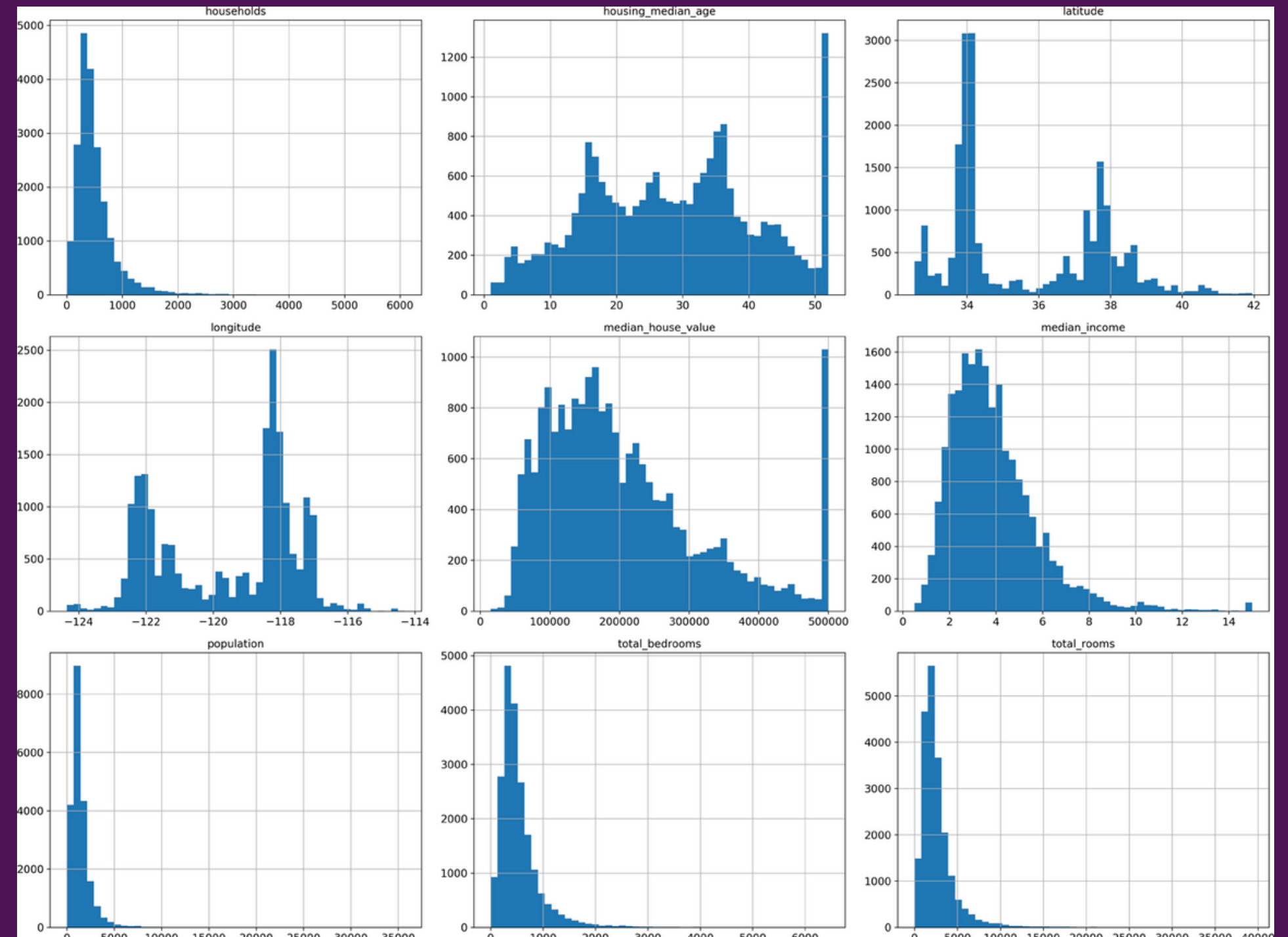


Fig: Histogram for each attribute

Preparing data for Machine Learning Algorithm

Data Cleaning

ML algorithms cannot work with missing features.

- Get rid of whole feature.
- Set the values to some value (zero, mean, median)

Handling text and categorical attributes

ML algorithms prefers to work with numbers rather than text or strings.

Feature Scaling

Two common ways to get all attributes to have the same scale.

- MinMax Scaling
- Standardization

Select and Train a Model



We framed the problem, got the data and explored it, we sampled a training set and a test set, and we wrote transformation pipelines to clean up and prepare our data for Machine Learning Algorithms. We are now ready to select and train a machine learning algorithms.

Fine-tune the Model

GridSearch CV

One way to that that with the hyperparameter values manually.

Instead we can use Scikit Learn GridSearch CV.

GridSearchCV as a method for systematically searching through a grid of hyperparameter values.

Randomized Search

We can also use Randomized Search.

RandomizedSearchCV as a more efficient alternative, especially for large parameter spaces.

randomly sample from a predefined distribution, exploring a wider range of values.

Evaluate system on Test Set

Evaluate the final model on the test set.

Run full_pipelines to transform the data (transform(), fit_transform()) and evaluate the final model on test set.

```
final_model = grid_search.best_estimator_  
  
X_test = strat_test_set.drop("median_house_value", axis=1)  
y_test = strat_test_set["median_house_value"].copy()  
  
X_test_prepared = full_pipeline.transform(X_test)  
  
final_predictions = final_model.predict(X_test_prepared)  
  
final_mse = mean_squared_error(y_test, final_predictions)  
final_rmse = np.sqrt(final_mse)    # => evaluates to 47,730.2
```

Fig: Model Evaluation

Launch Monitor and Maintain System

Real-time monitoring during launch and system operation.

Key metrics or data points being monitored (e.g., system health, performance indicators, error logs).



Fig: A model deployed as a web service and used by a web application



Thank You